Pizza 42 — How It All Works

A complete walkthrough of the architecture, the design decisions, and what happens behind the scenes for every user action.

What's in this doc

1. The big picture in one paragraph

Pizza 42 is a single-page React app for ordering pizza. We do not store customer credentials — Auth0 does. The user logs in through Auth0's Universal Login (email/password, Google, or passkey). After login, our React app calls our own backend at `/api/orders` to place an order. The backend verifies the user's token, checks they have the right permission, confirms their email is verified by calling Auth0's Management API, then saves the order — also through the Management API — onto the user's record at Auth0. The next time the user logs in (or even just switches browser tabs), an Auth0 Action runs and copies the order history into the user's ID token as a custom claim. The React app reads the claim and renders the history. No database of our own, no passwords, no session cookies — Auth0 is the source of truth for identity *and* a stand-in customer database for this PoC.

2. The cast of characters

Component	Type	What it does	Why it's there
React SPA	Public client	The Pizza 42 UI customers interact with. Initiates Universal Login, holds tokens, calls the backend.	Single-page apps are how modern customer-facing apps are built. Easy to host, fast to load.
Pizza 42 (Auth0 SPA app)	OAuth client record	Tells Auth0 what's allowed: which redirect URLs, which grants, what scopes.	Auth0 needs an entry for every client that talks to it. This is the SPA's identity in Auth0.
Pizza 42 API	OAuth resource server record	Declares the API's identifier (`https://pizza42-api`) and which scopes it protects (`create:orders`).	Without this record, Auth0 won't issue meaningful access tokens — they'd be opaque `/userinfo` tokens not usable by our backend.
Vercel serverless backend	Resource server (the actual code)	Validates user JWTs, enforces scope, checks `email_verified` , persists orders and profile via Mgmt API.	This is where real authorization decisions live. Client-side checks are advisory; the server is authoritative.
Pizza 42 Backend (M2M app)	OAuth confidential client	The backend's *own* identity in Auth0. Used to call the Management API. Holds a client secret.	The SPA can't call the Management API directly — it's too privileged. The M2M app gives the backend its own credentials for server-only operations.
Google social connection	Identity provider	Lets users sign in with their Google account.	One-click signup, pre-verified email. Removes a step from the user's first experience.
Database connection	Identity provider	Email/password and passkeys. Auth0 stores the hashed credentials.	Required for users who don't want to use

			Google. Passkeys give us the password-free experience.
Post-Login Action	Custom Node.js code	Runs on every login. Reads `app_metadata.orders` and `user_metadata.marketingProfile` and adds them as custom claims on the ID token.	Auth0 strips non-standard claims from tokens unless code explicitly adds them. The Action is the supported extensibility point. Rules and Hooks are deprecated.

> **Mental shortcut:** Three OAuth clients (SPA, M2M, API record), two identity providers (Google + Database), one Action. Five moving parts.

3. Three tokens, three jobs

The most important concept in this entire system is that **tokens are not interchangeable**. Three different tokens are issued, each for a different consumer.

Token	Format	Who consumes it	What it's for	Where it lives
ID Token	JWT (OIDC standard)	The React SPA	"Who is the user?" — name, email, picture, `email_verified`, custom claims (orders, marketing profile).	localStorage in the browser. Never sent to our backend.
Access Token	JWT (RS256-signed)	Our backend API	"What can the user do?" — bearer credential the backend verifies. Carries the `scope` claim.	localStorage in the browser. Sent in `Authorization: Bearer` header to `/api/orders` and `/api/profile`.
Refresh Token	Opaque string	Auth0's `/oauth/token` endpoint	"Get me fresh tokens without making the user log in again." Used by the SPA when the tab regains focus or the access token nears expiry.	localStorage in the browser. Sent only to Auth0, never to our backend.

Why three?

- **Different audiences.** ID token `aud` = the SPA's client ID. Access token `aud` = the API identifier. Mixing them violates "one token per resource."
- **Different sensitivity.** The refresh token is the keys-to-the-kingdom and should never be sent anywhere except Auth0's token endpoint.
- **Different lifetimes.** Access tokens are short (~24h default). ID tokens are session-length. Refresh tokens can live for weeks (with rotation, each use invalidates the previous one).

What's in our access token (default)

```
{
  "iss": "https://apnagra.us.auth0.com/",
  "sub": "google-oauth2|117679124118974227110",
  "aud": "https://pizza42-api",
  "azp": "PXtYKMLelP3t54ylgplcij8uKYfrqo52",
  "scope": "openid profile email create:orders",
  "iat": 1748373447,
  "exp": 1748459847,
  "gty": "authorization_code"
}
```

Note: *no* identity data — no email, no name, no `email_verified`. By design.

What's in our ID token (after Action runs)

```
{
  "iss": "https://apnagra.us.auth0.com/",
  "sub": "google-oauth2|117679124118974227110",
  "aud": "PXtYKMLelP3t54ylgplcij8uKYfrqo52",
  "name": "Amarpreet Nagra",
  "email": "apnagra24@gmail.com",
  "email_verified": true,
  "picture": "https://...",
  "https://pizza42.com/orders": [...],            ← added by Action
  "https://pizza42.com/marketing_profile": {...}, ← added by Action
  "iat": 1748373447,
  "exp": 1748459847
}
```

4. The user journeys

Journey 1 — First-time signup with email/password

```
User clicks "Sign up"
  → SPA calls loginWithRedirect({ screen_hint: "signup" })
  → Browser redirects to Auth0 Universal Login (signup form)
  → User submits email + password
  → Auth0 creates user record (email_verified = false)
  → Auth0 emails verification link
  → Auth0 redirects to http://localhost:3000/?code=...
  → SPA exchanges code + PKCE verifier for tokens
  → SPA loads dashboard
       - Order button: DISABLED (banner says "verify email")
       - History: empty
```

Journey 2 — Verifying email mid-session

```
User opens email in another tab, clicks verification link
  → Auth0 flips user's email_verified = true in the database
       (The cached token in the Pizza 42 tab still says false.)
User switches back to Pizza 42 tab
  → window 'focus' event fires
  → useEffect handler calls getAccessTokenSilently({ cacheMode: "off" })
  → SDK POSTs the refresh token to /oauth/token
  → Auth0 RE-RUNS the Post-Login Action
  → Auth0 issues new ID + access tokens with email_verified = true
  → SDK swaps cached tokens in localStorage
  → React re-renders
       - Banner disappears
       - Order button: ENABLED
```

Journey 3 — Placing an order

```
User picks "Margherita" and clicks "Place Order"
  → SPA calls getAccessTokenSilently() → returns cached access token
  → SPA → POST /api/orders
       Headers: Authorization: Bearer <access token>
       Body: { item: "Margherita", price: 12 }

Backend (api/orders.js):
  1. jwtVerify(token) — checks signature, issuer, audience, expiry
  2. payload.scope.includes("create:orders") — authorization check
  3. getManagementToken() — Client Credentials grant
       POST /oauth/token { client_id, client_secret, grant_type: "client_credentials"
       → returns M2M access token (cached in memory)
  4. GET /api/v2/users/{sub} with M2M token
       → returns user record { email_verified, app_metadata: { orders: [...] }, ... }
  5. if (!userRecord.email_verified) return 403
  6. orders = [...existing, newOrder]
  7. PATCH /api/v2/users/{sub} { app_metadata: { orders } }
  8. Return 200 { order, orders }

User sees "Order placed!" but history doesn't update yet — the cached
ID token doesn't have the new order. Same fix as Journey 2: switch tabs.
```

Journey 4 — Editing the marketing profile

```
User fills out "Tell us about you" → clicks Save
  → SPA → POST /api/profile { favoriteItem, dietary, ... }

Backend (api/profile.js):
  1. jwtVerify(token) — same JWT check
       (No scope check — anyone logged in can edit their own profile)
  2. Sanitize input — whitelist allowed fields
  3. getManagementToken() — reuse cached if still valid
  4. GET user → read existing marketingProfile
  5. Merge: marketingProfile = { ...existing, ...incoming, updatedAt: now }
  6. PATCH user { user_metadata: { marketingProfile } }
  7. Return 200

User switches tabs, returns → refresh-token call → Action re-runs →
new ID token has updated marketing_profile claim → UI shows new values.
```

Journey 5 — Logging in with Google

```
User clicks "Continue with Google"
  → Auth0 redirects to Google OAuth consent
  → User approves
  → Google returns to Auth0 with profile data
  → Auth0 creates/updates user (sub = "google-oauth2|...")
      email_verified = true (Google vouches for it)
  → Post-Login Action runs (orders + marketing_profile injected)
  → Auth0 redirects to Pizza 42 with code
  → SPA exchanges code for tokens
  → User lands on dashboard with email already verified.
```

5. The data system — where state lives and how it moves

Three buckets, three owners

Bucket	Lives in	Owner	What we store there
`app_metadata`	Auth0 user record	Backend only	`orders` — system-owned, tamper-proof
`user_metadata`	Auth0 user record	Backend or user (via specific scope)	`marketingProfile` — user-editable preferences
Standard profile	Auth0 user record	Auth0 / identity provider	name, email, picture, `email_verified` — set by Auth0 itself

Why orders go in `app_metadata`, not `user_metadata`

- Auth0 exposes a scope called `update:current_user_metadata` that a SPA can request to let users edit their own `user_metadata` directly from the browser.
- There is **no equivalent** scope for `app_metadata` — it can only be written via a backend with M2M credentials.
- Orders are *system-owned* facts: the kitchen creates them, not the user. They must be tamper-proof.
- By putting orders in `app_metadata`, even if a future engineer adds the `update:current_user_metadata` scope (e.g. to let users edit their saved address), order history stays safe.

Why PATCH replaces, doesn't merge

This is a subtle gotcha. The Auth0 Management API's PATCH operation *replaces* sub-objects rather than deep-merging. If our backend sent `{ app_metadata: { orders: [...] } }` without first reading the existing `app_metadata`, we'd wipe out any other fields stored there.

So every write follows a read-modify-write pattern:

```
const userRecord = await getUser(sub, mgmtToken);
const existing = userRecord.app_metadata?.orders ?? [];
const orders = [...existing, newOrder];
await patchUser(sub, {
  app_metadata: { ...userRecord.app_metadata, orders }
}, mgmtToken);
```

> **Race condition:** two concurrent orders both read the same `existing`, both append, second write wins → one order lost. In production we'd use a real database with transactional writes. Mentioning this in the panel demonstrates depth.

6. The Action — how the token gets shaped

The Auth0 Post-Login Action is the only place in our system that can modify the ID token. Tokens are signed by Auth0's private key — the SPA and our backend can read tokens but can never alter them without breaking the signature.

The full Action code

```
exports.onExecutePostLogin = async (event, api) => {
  const orders = event.user.app_metadata?.orders ?? [];
  const marketing = event.user.user_metadata?.marketingProfile ?? null;
  api.idToken.setCustomClaim("https://pizza42.com/orders", orders);
  api.idToken.setCustomClaim(
    "https://pizza42.com/marketing_profile",
    marketing,
  );
};
```

When does it run?

- Every fresh login (email/password, social, passkey)
- Every refresh-token exchange (this is why the focus listener works — it triggers a refresh-token call, which re-runs the Action)

- NOT on the order placement HTTP request — Actions only fire on auth lifecycle events, not on arbitrary API calls

Why claims must be namespaced

OIDC reserves the short names (`name`, `email`, `sub`, `iat`, etc.) for standard or future use. Custom claims must be URI-namespaced or Auth0 will **silently strip them** — no error, no warning. That's why our claim names look like URLs: `https://pizza42.com/orders`. The URL doesn't have to resolve; it's just a unique label.

> **The silent-drop trap:** If you forget the namespace, the Action runs successfully, the token is issued, the claim just isn't there. One of the most time-wasting bugs in Auth0. Always namespace, always check the decoded token after deploying an Action.

Rules and Hooks are dead

Feature	Status
Actions (what we use)	Current
Rules	EOL Nov 18, 2026 — unavailable to tenants created after Oct 16, 2023
Hooks	Same EOL

Half the older public Pizza 42 reference repos still demonstrate Rules. Copying those without modernizing would be an instant credibility red flag for an identity-specialist role.

7. The M2M app — the privileged second identity

The Pizza 42 Backend (M2M) app exists for one reason: to give our backend its own credentials to call Auth0's Management API. Without it, the backend could verify user tokens but couldn't read or write user records.

Why a separate app, not part of the SPA

- **Credential isolation.** The SPA cannot safely hold a client secret — anyone can view its JavaScript. The M2M app's secret lives in server-only env vars.
- **Different grant.** The SPA uses Authorization Code + PKCE (user is present). The M2M app uses Client Credentials (no user). Auth0 doesn't allow mixing grants in one app record.
- **Least privilege.** The M2M app is authorized for specific Management API scopes (`read:users`, `update:users_app_metadata`, etc.). It has no other powers. Leaking its credentials would let an

attacker modify user records but not, say, change tenant settings.

- **Audit clarity.** Auth0 logs every Management API call and tags it with the app that made it. Mixing the SPA and the backend would muddy investigations.

The Client Credentials grant

```
POST https://apnagra.us.auth0.com/oauth/token
{
  "client_id": "...",
  "client_secret": "...",
  "audience": "https://apnagra.us.auth0.com/api/v2/",
  "grant_type": "client_credentials"
}

→ Returns a Management API access token (~24h lifetime)
```

We cache the returned token in memory in the serverless function. The first order in a fresh function instance pays the cost of one extra round-trip; subsequent orders skip it.

8. Every design decision and why we made it

Decision	Why
Universal Login (not embedded)	Pizza 42 never touches passwords. MFA, password reset, social, passkeys are all turn-key. Tradeoff: less branding control (mitigated by Auth0's customization options).
Authorization Code + PKCE (not Implicit or ROPC)	Only safe grant for browser apps. PKCE protects against auth-code interception. Implicit puts tokens in URLs (deprecated). ROPC requires the app to see the password (defeats the whole point).
Refresh tokens with rotation	Iframe silent renew is blocked by third-party cookie restrictions in Chrome/Safari/Brave. Refresh tokens use back-channel POSTs — work everywhere. Rotation invalidates the previous token on each use; reuse signals a breach.
Plain scopes (not RBAC)	Single-persona PoC. `create:orders` is enough. Easy to upgrade later — backend already checks both `scope` and `permissions` claims.
Orders in `app_metadata`	System-owned, tamper-proof. `user_metadata` has the `update:current_user_metadata` escape hatch; `app_metadata` doesn't.
Marketing profile in `user_metadata`	User-owned data (favorite pizza, dietary prefs). Users should be able to edit their own preferences.
`email_verified` via live Mgmt API lookup (not token claim)	The token would be a stale snapshot from login. A user who verifies mid-session would be blocked until they re-login. The live lookup costs one extra HTTP call (which the backend was making anyway to save the order) and gives always-current state.
Tab-focus listener for silent re-auth	Refresh tokens don't auto-fire on their own — they fire on demand. A focus listener calls `getAccessTokenSilently({ cacheMode: "off" })` when the user returns to the tab, picking up any state changes (verified email, new orders, updated profile).
Actions over Rules	Rules are EOL. New tenants can't even use them.
Namespaced custom claims	Auth0 silently strips non-namespaced claims. The namespace doesn't need to resolve.

9. The gotchas we hit (and how we fixed them)

Symptom	Root cause	Fix
`Callback URL mismatch`	Auth0 SPA app didn't have `http://localhost:3000` in Allowed Callback URLs	Add it. The SDK uses `window.location.origin`, not `/callback`.
`Service not found: https://pizza42-api`	The API record didn't exist in Auth0	Create the API with that exact identifier.
`Client is not authorized to access resource server`	API used per-app authorization, SPA wasn't granted access	API → Application Access tab → grant the SPA + the `create:orders` scope.
`Missing required scope: create:orders`	Scope wasn't checked in API Access tab for the SPA	Same fix — explicit permission grant on API Access tab.
`401 fetch failed` in backend logs	Backend env vars weren't loaded by `vercel dev` after linking	Add env vars to the Vercel project via `vercel env add`.
Order history doesn't update after placing an order	ID token cached in localStorage doesn't include the new order	Focus listener + refresh token + Action re-run on every silent renew.
Button stays disabled after verifying email	Same staleness — cached token says `email_verified=false`	Same fix — focus listener.
Marketing profile claim missing from token	Action wasn't redeployed after we added the second claim line	Update Action body in dashboard → click Deploy.
Google profile picture broken on the page	Google blocks `lh3.googleusercontent.com` requests with certain Referer headers	Add `referrerPolicy="no-referrer"` to the `<img>`. Fall back to initials.
Consent screen every login	Auth0 forces user consent on `localhost` as anti-phishing	Disappears automatically on the deployed HTTPS URL.

10. What this PoC does differently from a textbook OAuth demo

Anyone can wire up Universal Login and call it done. The interesting parts are where we made considered choices, often pushing back on the brief.

We disagreed with the spec on three points

1. The PDF said "put orders in the ID token." We did — but capped at the natural size where it stops being a token concern and starts being a database concern. In production we'd put only a summary (`orderCount` , `lastOrderAt`) in the token and fetch the full list from `/api/orders` .
2. The PDF said "save orders to user's Auth0 profile." We did, but in `app_metadata` (tamper-proof) not the more obvious `user_metadata` . The reasoning is real: in production the source of truth would be our own database, with Auth0 mirroring only a thin summary.
3. The PDF implied a client-side email-verified gate. We did that for UX, but the actual security gate is on the backend via a live Mgmt API lookup — not a token claim, which would be stale.

We built the mid-session freshness fix

Most demos require a logout/login cycle to pick up state changes. We added a focus listener that triggers a refresh-token call when the user returns to the tab. The Action re-runs, the ID token refreshes, the UI updates — all without the user doing anything. "*The token is the cache, refresh-token-on-focus keeps it warm.*"

We chose Actions on purpose

We didn't just use Actions because that's what we know — we chose them because Rules are EOL and unavailable to new tenants. Calling this out signals platform awareness.

We considered passkeys as a deliberate product decision

Pizza 42 is impulse-buy. Every friction point costs conversion. Passkeys eliminate password entry on supported devices and are phishing-resistant by design — they hit security and UX at the same time. Not just a "checkbox enabled" feature.

11. Glossary

Term	Meaning
SPA	Single-Page Application. Our React frontend.
M2M	Machine-to-Machine. An OAuth client with no user — used for server-to-server calls.
OIDC	OpenID Connect. The identity layer built on top of OAuth 2.0. Defines the ID token and standard claims.
JWT	JSON Web Token. A self-contained, cryptographically signed token format.
JWKS	JSON Web Key Set. The public keys Auth0 publishes so our backend can verify token signatures.
RS256	RSA-SHA256. The asymmetric signing algorithm — Auth0 signs with private key, we verify with public key from JWKS. No shared secret.
PKCE	Proof Key for Code Exchange. An extension to Authorization Code that protects public clients (SPAs) from code-interception attacks. Replaces the client secret with a one-time crypto challenge.
Authorization Code grant	The OAuth flow where the user is redirected to the IdP, authenticates, and is sent back with a short-lived code that the client exchanges for tokens.
Client Credentials grant	OAuth flow for M2M — client identifies itself with id+secret, no user involved.
Resource server	OAuth term for an API protected by access tokens. In our case, our backend at `/api/orders`.
Audience	The `aud` claim in a token. Tells the resource server "this token is meant for you, not someone else."
Scope	A permission string. Our API declares `create:orders`. The token's `scope` claim lists which scopes were granted.
Action	Auth0's current extensibility model. Node.js code that runs at specific points in the auth lifecycle.
Universal Login	Auth0-hosted login page. The app redirects to Auth0; users authenticate there and are sent back. Pizza 42 never sees credentials.
Front channel	Communication via browser redirects (visible in URLs). Used for auth code exchange.

Back channel	Server-to-server HTTP requests. Used for token exchange, JWKS fetch, Management API calls.
Refresh token rotation	Each refresh issues a new refresh token and invalidates the previous one. If the old one is ever reused, the entire family is revoked (breach signal).
Namespaced claim	Custom JWT claim with a URI-shaped name (e.g. `https://pizza42.com/orders`) so it doesn't collide with reserved OIDC claims.

> **How to read this doc:** Sections 1–3 are the "what." Sections 4–5 are the "how." Sections 6–8 are the "why." Sections 9–10 are the "what makes this PoC defensible in an interview." Section 11 is your safety net for vocabulary.